

ASSIGNMENT-13.4

NAME: E.NIKHIL

H.NO:2303A51847

B-13

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Expected Output

```
[4, 8, 12, 16, 20]
```

Prompt:

#Refactor this loop-based Python code into a shorter, Pythonic version using a list comprehension keeping the same output.

Original code:

```
#Refactor this loop-based Python code into a shorter, Pythonic version using a list comprehension keeping the same output.
# Original code:
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Refactored code:

```
# Refactored code using a list comprehension:
values = [2, 4, 6, 8, 10]
doubled = [v * 2 for v in values]
print(doubled)
```

Output:

```
[4, 8, 12, 16, 20]
[4, 8, 12, 16, 20]
```

Justification :

The refactored version using a list comprehension is more efficient because it eliminates the need for repeatedly appending values inside a loop. In Python, list comprehensions are optimized for performance and execute faster than manual iteration, especially when dealing with larger datasets. This makes the transformation more suitable for data preprocessing tasks where speed and memory usage matter.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

Task Description

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

Legacy Code

```
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Expected Output

Refactoring with AI improves quality

Prompt:

#Refactor this Python code that builds a sentence using repeated string concatenation. Make it more efficient and readable with join, while keeping the same output.

Original code:

```
#Refactor this Python code that builds a sentence using repeated string concatenation. Make it more efficient and readable with join, while keeping the same output.
# Original code:
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = ""
for w in words:
    message += w + " "
print(message.strip())
```

Refactored code:

```
# Refactored code:
words = ["Refactoring", "with", "AI", "improves", "quality"]
message = " ".join(words)
print(message)
```

Output:

```
Refactoring with AI improves quality
Refactoring with AI improves quality
```

Justification:

The join method also improves readability by clearly expressing the intent: combine words with spaces. This eliminates the need for extra steps like trimming whitespace with strip, and future readers can immediately understand the purpose of the code without tracing through loop logic. It makes the code concise, clean, and easier to maintain.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

Task Description

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Expected Output

Default timeout used

Prompt:

#Refactor this Python code that checks for dictionary keys using if-else. Use a safer and shorter approach like dict.get() to handle missing keys, while keeping the same output.

Original code:

```
#Refactor this Python code that checks for dictionary keys using if-else. Use a safer and shorter approach like dict.get() to handle missing keys, while keeping the same
# Original code
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Refactored code:

```
# Refactored code
config = {"host": "localhost", "port": 8080}
print([config.get("timeout", "Default timeout used")])
```

Output:

```
Default timeout used
Default timeout used
```

Justification:

Using `dict.get()` makes the code safer because it directly handles missing keys without requiring explicit checks. Instead of risking a `KeyError` or writing extra conditional logic, the `get` method allows you to provide a default value that will be returned if the key is absent. This ensures the program behaves predictably across different deployment settings.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Task Description

- Examine the multiple `if–elif` conditions used to determine operations.
- Ask AI to suggest a cleaner, scalable alternative.
- Refactor the logic using mapping techniques while preserving functionality.

Legacy Code

```
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Expected Output

5.0

Prompt:

#Refactor this Python code that uses multiple `if–elif` conditions for operations. Use a dictionary mapping approach to make it cleaner and scalable, while keeping the same output.

Original code:

```
#Refactor this Python code that uses multiple if-elif conditions for operations. Use a dictionary mapping approach to make it cleaner and scalable, while keeping the same
#Original code:
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = "None"
print(result)
```

Refactored code:

```
#Refactored code:
action = "divide"
x, y = 10, 2
operations = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b
}
result = operations.get(action, lambda a, b: "None")(x, y)
print(result)
```

Output:

```
5.0
5.0
```

Justification:

Using a mapping structure improves readability by clearly associating each action with its corresponding function. Instead of scanning through a long chain of conditions, readers can quickly see all available operations in one place. This design is cleaner, avoids repetitive logic, and makes the intent of the code immediately obvious.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Task Description

- Identify the explicit loop used for searching an item.
- Use AI assistance to refactor the logic into a more concise and readable form.
- Maintain the same output behavior.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

Prompt:

#Refactor this Python code that uses a manual loop to search for an item in a list. Use a more concise approach like the in operator, while keeping the same output.

Original code:

```
#Refactor this Python code that uses a manual loop to search for an item in a list. Use a more concise approach like the in operator, while keeping the same output.
#original code:
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Refactored code:

```
#refactored code:
inventory = ["pen", "notebook", "eraser", "marker"]
item_to_find = "eraser"
print("Item Available" if item_to_find in inventory else "Item Not Available")
```

Output:

```
Item Available
Item Available
```

Justification:

The `in` operator also improves readability by expressing the intent in a straightforward way: *check if "eraser" exists in the inventory*. The legacy code requires a flag variable (`found`) and a `break` statement, which makes the logic longer and harder to follow. The refactored version condenses the logic into one clear line, eliminating unnecessary complexity. This concise style makes the code easier to understand at a glance and reduces the chance of errors, which is especially valuable when multiple developers are maintaining the same codebase.